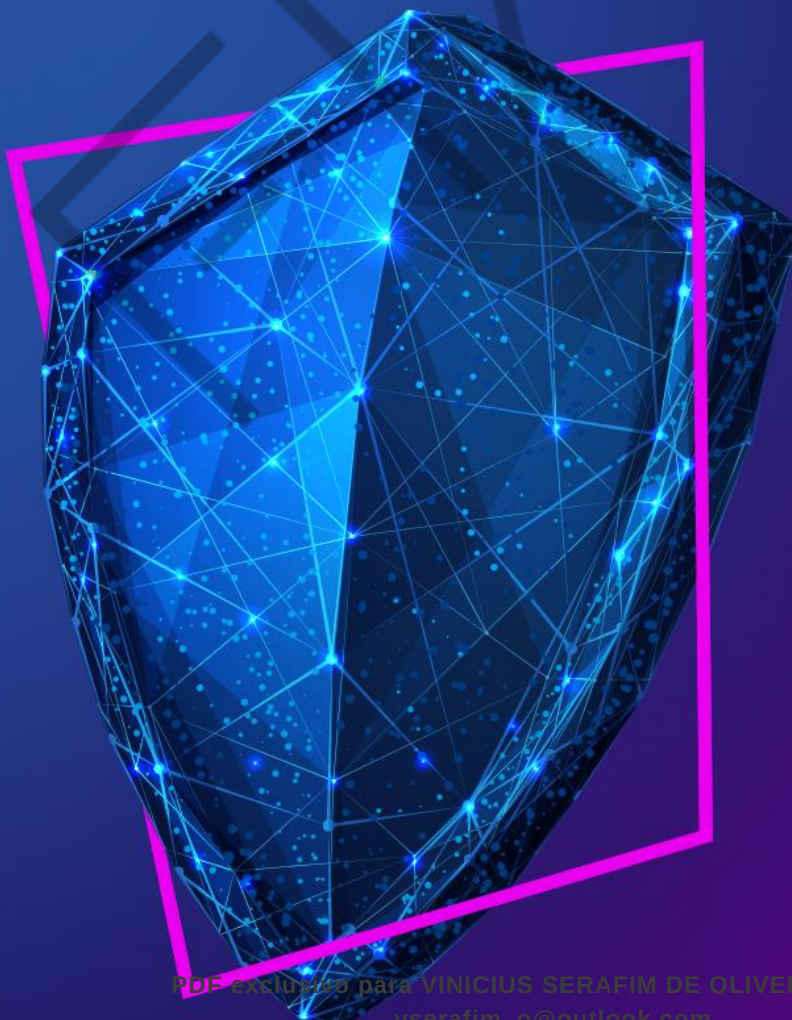


BUILD YOUR BASES FIRST

CRIPTOGRAFIA: O SEGREDO DE CESAR



2

LISTA DE FIGURAS

Figura 1 – Reconstrução de uma cítala grega, usada supostamente por milhares espartanos.....	6
Figura 2 – Cifra de César	8
Figura 3 – Quadrado de Vigenère	8
Figura 4 – Cifra de Vigenère	9
Figura 5 – Cifragem da mensagem por Vigenère.....	9
Figura 6 – Máquina Enigma, versão da Marinha, exposta em Bletchley Park	10
Figura 7 – <i>Sniffing attack</i>	12
Figura 8 – <i>Man-in-the-middle attack (MITM)</i>	12
Figura 9 – Homem acorrentado a uma maleta com a chave única	15
Figura 10 – Criptografia simétrica (chave privada).....	15
Figura 11 – Garantia da CONFIDENCIALIDADE	16
Figura 12 – Garantia da AUTENTICIDADE.....	17
Figura 13 – Algoritmo 3DES.....	18
Figura 14 – Algoritmo de Diffie-Hellman.....	21
Figura 15 – <i>Hashes</i> do arquivo /var/log/messages	25
Figura 16 – <i>Hashes</i> do Hash_ID.	27
Figura 17 – Extração do <i>hash</i> da <i>string</i>	27
Figura 18 – Verificação do <i>hash</i> MD5	28
Figura 19 – Verificação do <i>hash</i> SHA1	29
Figura 20 – Cifragem de transposição	30
Figura 21 – <i>Shell script</i> para codificação/decodificação em base64	31
Figura 22 – Codificação do <i>input</i> do usuário	31

LISTA DE TABELAS

Tabela 1 – Tabela de cifragem por substituição	7
Tabela 2 – Principais características dos algoritmos criptográficos	20
Tabela 3 – Propriedades dos algoritmos de <i>hash</i> seguros	25

EMSE

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Baixando o Hash ID usando o comando <code>wget</code>	26
Comando de prompt 2 – Comando <code>echo</code> com <code>md5sum</code>	27
Comando de prompt 3 – Hash_ID no <i>python</i>	28

EMANIP

SUMÁRIO

CRIPTOGRAFIA: FUNDAMENTOS E APLICAÇÕES.....	6
1 INTRODUÇÃO	6
2 A CIFRA DE CÉSAR.....	7
3 A CIFRA DE VIGENÈRE.....	8
4 A ORIGEM DO ENIGMA	9
4.1 Enigma e a Segunda Guerra Mundial	9
5 CONCEITUANDO CRIPTOGRAFIA	10
6 SISTEMAS CRIPTOGRÁFICOS	14
6.1 Sistema de chave privada (criptografia simétrica).....	14
6.2 Sistema de chave pública (criptografia assimétrica)	16
7 ALGORITMOS CRIPTOGRÁFICOS	17
7.1 Algoritmos criptográficos simétricos	18
7.1.1 DES e 3DES	18
7.1.2 RC2, RC4 e RC5.....	19
7.1.3 Idea (<i>International Data Encryption Algorithm</i>).....	19
7.1.4 <i>Blowfish</i>	20
7.1.5 AES	20
7.2 Algoritmos criptográficos assimétricos	21
7.2.1 Algoritmo de Diffie-Hellman.....	21
7.2.2 RSA.....	22
7.2.3 SSL / TLS.....	22
7.2.4 DSA.....	23
8 ALGORITMOS DE <i>HASHING</i>	23
9 TIPOS DE CIFRAS	29
9.1 Cifras de transposição.....	29
9.2 Base64	30
REFERÊNCIAS.....	33

CRIPTOGRAFIA: FUNDAMENTOS E APLICAÇÕES

1 INTRODUÇÃO

A criptografia (que vem do grego *kryptós*, que significa "escondido" e *gráphein*, palavra grega para "escrita") é a prática milenar de comunicação segura na presença de terceiros (ou adversários), embaralhando uma mensagem escrita em uma mensagem inteligível e desembaralhando-a de forma a reaver a mensagem original. Desta maneira, se os mensageiros fossem interceptados as mensagens originais permaneceriam secretas.

Antes do advento da matemática avançada, ciências da computação e comunicação, engenharia elétrica e física (que compõem a criptografia moderna), as soluções eram mais simples, mas engenhosas. Na Figura “Reconstrução de uma cítala grega, usada supostamente por milhares espartanos” podemos observar uma cítala, um bastão que supostamente ficava em posse de militares espartanos; as mensagens eram gravadas em peles de carneiro juntamente com diversos outros caracteres aleatórios, assim sendo, apenas com uma cítala de mesmo comprimento e espessura os caracteres certos seriam realinhados, revelando a mensagem original.



Figura 1 – Reconstrução de uma cítala grega, usada supostamente por milhares espartanos
Fonte: WIKIMEDIA COMMONS (2007)

Também utilizadas há milênios, as cifras de substituição são as primeiras cifras conhecidas. A técnica é simples, porém elegante: os caracteres do texto original eram simplesmente substituídos por outros caracteres, números ou símbolos, por meio de

uma tabela de substituição; tome como exemplo a substituição de cada letra do alfabeto por um número, como mostra a tabela “Tabela de cifragem por substituição”.

Tabela de substituição																									
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	X	W	Y	
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	
Mensagem em texto claro:								C	R	I	P	T	O	G	R	A	F	I	A						
Texto cifrado: 3-18-9-16-20-15-7-18-1-6-9-1																									

Tabela 1 – Tabela de cifragem por substituição
Fonte: Elaborada pelo autor (2018)

Observe-se que a ordem dos símbolos de substituição, no caso, os números de 1 a 26 podem assumir qualquer ordem desejada: quanto mais aleatória, maior a força da cifragem.

2 A CIFRA DE CÉSAR

Segundo o historiador romano Suetônio (121), Júlio César prezava muito por sua privacidade em suas correspondências pessoais e, em cartas para Cícero, quando César tinha algo confidencial a dizer este o fazia em código, empregando uma cifra de substituição que, posteriormente, foi atribuída a ele.

A **cifra de César é uma cifra de substituição monoalfabética** na qual cada letra do texto original é substituída por outra com deslocamento predeterminado, por exemplo, considere-se um deslocamento de 3, isto significa que todas as letras “a” da mensagem deverão ser substituídas pela letra “d”, todas as letras “b” da mensagem deverão ser substituídas pela letra “e” e assim sucessivamente (vide figura “Cifra de César”).

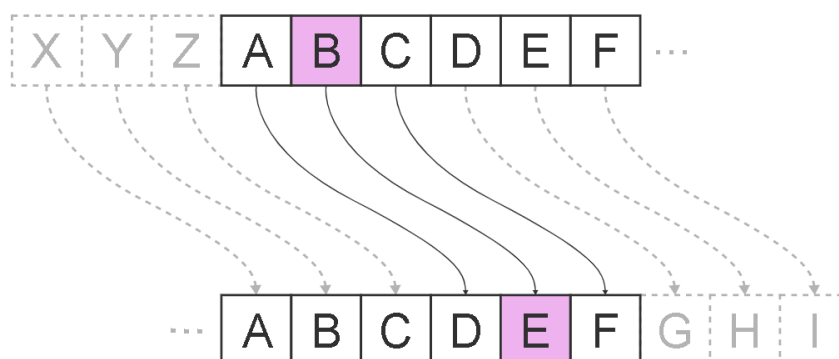


Figura 2 – Cifra de César
Fonte: Crypto (s.d.)

3 A CIFRA DE VIGENÈRE

A cifra de Vigenère, por sua vez, é considerada uma evolução da cifra de César e se trata de uma substituição polialfabética. Sua principal diferença recai sobre a forma como os caracteres são deslocados para cifragem da mensagem. Enquanto na cifra de César todos os elementos de uma mensagem têm o mesmo deslocamento, na cifra de Vigenère, o deslocamento de cada caractere é dado com base em uma senha - a chave criptográfica utilizada para codificação, e na posição do caractere em relação à mensagem. A figura abaixo ilustra a tabela polialfabética utilizada pela cifra de Vigenère.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Figura 3 – Quadrado de Vigenère
Fonte: Elaborada pelo autor (2018)

Para codificação da mensagem “criptografia simétrica”:

- Deve-se escolher a chave criptográfica a ser utilizada para mapeamento da mensagem no quadrado de Vigenère: urano.
- Igualar o comprimento da chave ao comprimento da mensagem:

c	r	i	p	t	o	g	r	a	f	i	a	s	i	m	e	t	r	i	c	a
u	r	a	n	o	u	r	a	n	o	u	r	a	n	o	u	r	a	n	o	u

Figura 4 – Cifra de Vigenère
Fonte: Elaborada pelo autor (2018)

Da correspondência estabelecida entre cada caractere da mensagem em texto claro e a chave criptográfica fornecida (tabela), obtém-se o par a ser utilizado para consulta ao quadrado de Vigenère e cifragem da mensagem.

Mensagem (linha)	c	r	i	p	t	o	g	r	a	f	i	a	s	i	m	e	t	r	i	c	a
Chave (coluna)	u	r	a	n	o	u	r	a	n	o	u	r	a	n	o	u	r	a	n	o	u
Caractere obtido no quadrado de Vigenère	w	i	i	c	h	i	x	r	n	t	c	r	s	v	a	y	k	r	v	q	u

Figura 5 – Cifragem da mensagem por Vigenère
Fonte: Elaborada pelo autor (2018)

Da tabela vem que a mensagem cifrada é **wiichixrntcrsvaykrvqu**.

4 A ORIGEM DO ENIGMA

4.1 Enigma e a Segunda Guerra Mundial

Enigma é o nome pela qual ficou conhecida a máquina eletromecânica de criptografia com rotores, extensamente utilizada pelos alemães na Segunda Guerra Mundial para cifrar códigos de guerra.



Figura 6 – Máquina Enigma, versão da Marinha, exposta em Bletchley Park
Fonte: Manske (2005)

Com o intuito de quebrar a criptografia das Enigmas utilizadas pela marinha alemã, um grupo de britânicos em Bletchley Park sob a liderança do matemático Alan Turing, criou a **Bomb**, o computador eletromecânico programável.

Graças a este esforço tecnológico e a descoberta dos segredos militares da marinha alemã, estima-se que a Segunda Guerra Mundial terminou um ano antes. Esta história é contada no filme “O Jogo da Imitação” (*The Imitation Game*), disponível no catálogo do Amazon Prime Vídeo.

5 CONCEITUANDO CRIPTOGRAFIA

Criptografia é uma das técnicas mais eficazes e difundidas para garantia da confidencialidade por meio da cifragem da mensagem, tornando-a, inicialmente, ilegível. Assim sendo, entende-se como:

- **Criptografar (também muitas vezes referenciado como encriptar)**, o processo de submissão da mensagem em texto puro (*clear text*) a um algoritmo específico que a codifica, tornando-a incompreensível.

- **Decriptografar (também muitas vezes referenciado como descriptar)**, a reversão do processo de criptográfico, restaurando a legibilidade e compreensão da mensagem original.
- *Clear text* o texto puro, explícito, com o qual a mensagem foi originalmente cunhada.
- Texto ou mensagem cifrada, como o resultado produzido pela submissão da mensagem ao algoritmo criptográfico.

Além da confidencialidade, a criptografia pode ser aplicada também a outros contextos, por exemplo, para garantia da autenticidade da mensagem, para verificação da integridade e não repúdio da mensagem, dentre outras possibilidades.

Pode-se citar como exemplos práticos da aplicação da criptografia, de forma transparente para o usuário a substituição do protocolo FTP pelo SFTP, a substituição do protocolo TELNET pelo SSH e a substituição do POP3 pelo POP3S. Em cada um dos três exemplos, os primeiros protocolos referidos (FTP, TELNET, POP3) trafegam suas mensagens em *clear text*, portanto, incapaz de garantir a confidencialidade da mensagem; enquanto as alternativas apontadas (SFTP, SSH e POP3S) utilizam-se de diferentes algoritmos criptográficos para garantir a confidencialidade da mensagem.

Ilustrando melhor o risco de serviços como os citados (FTP, TELNET, POP3) cuja comunicação acontece em texto puro, um atacante pode começar realizando o chamado *sniffing*, colocando sua interface de rede em modo promíscuo e assim capturar todos os pacotes por aquela rede trafegados. Observe que até mesmo as credenciais (como nome de usuário e senha) são transmitidos por estes serviços de forma aberta, ou seja, o atacante obtém tais informações com bastante facilidade.

Sniffing the networks

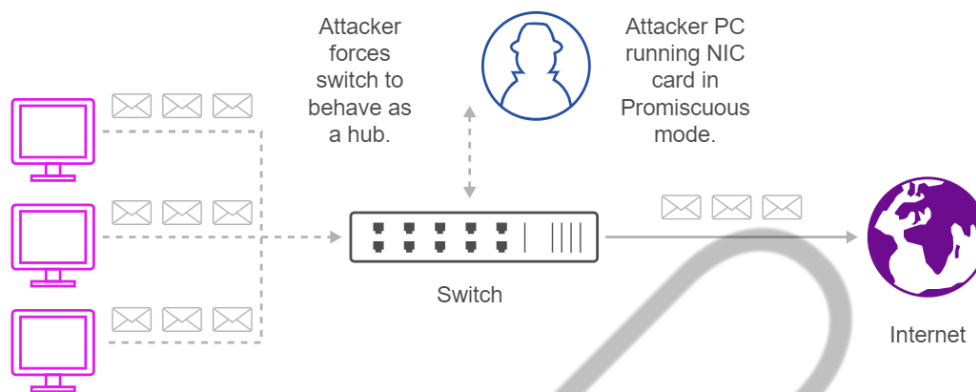


Figura 7 – *Sniffing attack*
Fonte: TUTORIALSPOINT (s.d.)

Posteriormente, o ataque pode evoluir para um ***man-in-the-middle attack (MITM)***, pela qual o atacante se posiciona no meio das comunicações entre os *hosts*, atuando como um intermediário invisível, podendo assim fazer adições maliciosas nos pacotes para seus objetivos escusos. Em uma variação mais grave, o invasor pode derrubar um dos *hosts* (geralmente o cliente do serviço) com um ataque de negação de serviços (*DoS attack*), tomando completamente seu lugar na comunicação com o servidor.

Man-in-the-middle attack

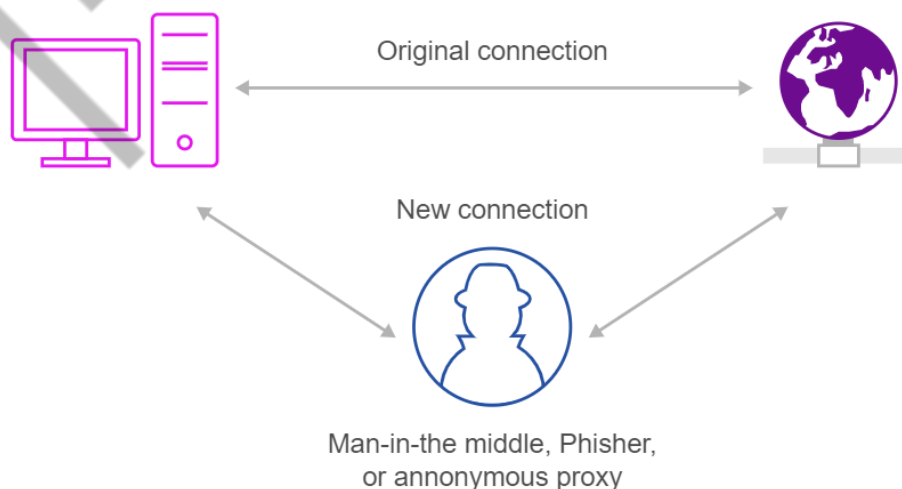


Figura 8 – *Man-in-the-middle attack (MITM)*
Fonte: TECHGNOR (s.d.)

Portanto, o uso de criptografia em serviços de rede é essencial para mitigar ataques como estes. Contudo, é importante que os algoritmos criptográficos sejam eficientes em relação ao uso dos recursos computacionais, como a capacidade de processamento do sistema, a fim de minimizarem o *overhead* decorrente dos processos de criptografia e decriptografia das mensagens.

Os sistemas criptográficos podem ser implementados de duas maneiras diferentes: via software, opção geralmente mais barata, entretanto, lenta se comparada à implementação baseada em *hardware*. Isso decorre da maior demanda de recursos computacionais, característica da criptografia por software, uma vez que a criptografia por *hardware* é feita por circuitos especializados, geralmente ASICs (*Application Specific Integrated Circuit*), circuitos integrados especializados que tratam o processo criptográfico de forma autônoma e independente da CPU do sistema, tornando-o assim mais caro, porém, mais rápido e com menos *overhead* para o sistema do que a implementação por software.

A FORTINET é um reconhecido *player* de mercado que costuma utilizar a criptografia por *hardware* em seus *appliances* para criação de VPNs.

Para que um algoritmo criptográfico possa ser considerado seguro, a descoberta da chave utilizada no processo de criptografia deve ser praticamente impossível de ser descoberta, assim como a decriptografia do texto cifrado, sem a chave que o criptografou.

Dois sistemas de chaves poderão ser utilizados: sistema de chave privada (ou chave secreta) ou sistema de chave pública. Adicionalmente, existe ainda o *message-digest*, mais conhecido como função *hash* (e a partir daqui referenciado simplesmente como *hash*), o qual não tem como objetivo a cifragem da mensagem, mas, sim, a verificação de sua integridade ou a criação de sua assinatura digital. As funções *hash* são caracterizadas por receber uma entrada de comprimento arbitrário, produzindo um *string* de saída de comprimento fixo e curto (por exemplo, 128 ou 160 bits).

Para que as funções *hash* sejam consideradas funcionais, elas devem apresentar duas propriedades:

- Baixo nível de colisão, o que significa que a probabilidade de duas entradas distintas produzirem o mesmo *hash* na saída deverá ser praticamente nula, mesmo que as entradas sejam muito semelhantes.

- Garantir que será praticamente impossível reconstituir a mensagem de entrada a partir do *hash* de saída.

6 SISTEMAS CRIPTOGRÁFICOS

Dois sistemas de chaves poderão ser utilizados pelos algoritmos criptográficos: sistema de chave privada (ou chave secreta) ou sistema de chave pública.

6.1 Sistema de chave privada (criptografia simétrica)

Sistema simples, utilizado para criptografia simétrica, ou seja, uma única chave criptografa e decriptografa a mensagem, sendo conhecida por todos os integrantes da comunicação.

Dada sua simplicidade, esses algoritmos são bastante rápidos, entretanto, dessa simplicidade surgem, ao menos, dois sérios problemas: o primeiro relacionado à forma como a chave criptográfica será distribuída entre os interlocutores, o que exige um canal seguro, uma vez que qualquer um que venha a possuir a chave, poderá decifrar a mensagem.

No passado, o canal seguro era o físico, e esta chave única era transportada em maletas acorrentadas em seus portadores, em cenas típicas de filmes de espionagem.



Figura 9 – Homem acorrentado a uma maleta com a chave única
Fonte: Shutterstock (2020)

E o segundo problema: a impossibilidade de garantir o não repúdio da mensagem. Simplificadamente, não repúdio significa impedir que um indivíduo ou entidade venha a negar a execução de alguma ação particular relacionada a uma mensagem ou transação, ou ainda, questionar a validade delas. A figura a seguir representa a criptografia simétrica.

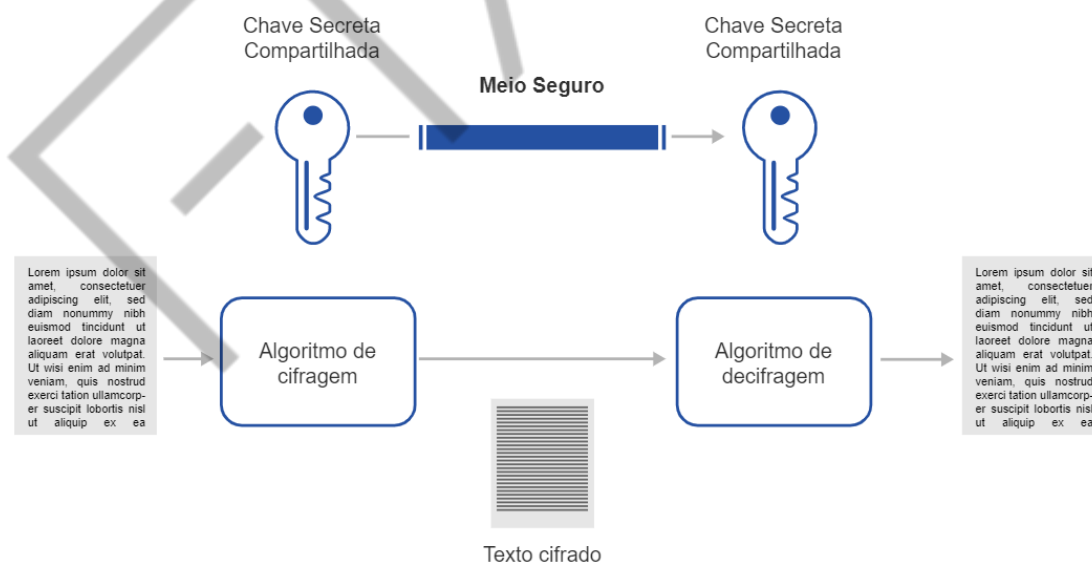


Figura 10 – Criptografia simétrica (chave privada)
Fonte: GTA UFRJ (2018)

6.2 Sistema de chave pública (criptografia assimétrica)

Criado em 1976 por Whitfield Diffie e Martin Hellman, este sistema utiliza duas chaves distintas, uma **pública** e outra **privada**. Embora notoriamente mais seguro que a criptografia simétrica e tendo ainda resolvido o problema de gerenciamento das chaves, é consideravelmente mais complexo e, portanto, mais lento que ela. A criptografia assimétrica constitui excelente mecanismo para garantir a confidencialidade e a autenticidade da mensagem. Considere-se o cenário ilustrado na figura abaixo: Beto é o **destinatário** de uma mensagem cifrada com sua chave pública.

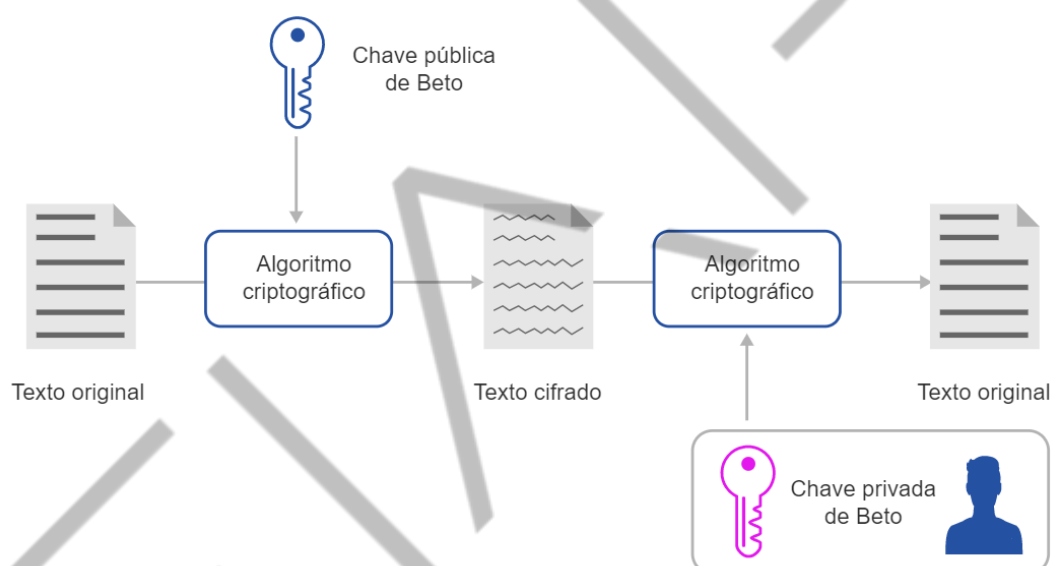


Figura 11 – Garantia da CONFIDENCIALIDADE
Fonte: Cartilha de Certificação Digital (2018)

Por meio de sua chave privada, que é secreta e não deve ser compartilhada com ninguém, somente Beto – destinatário da mensagem – será capaz de decifrá-la sendo assim assegurada a CONFIDENCIALIDADE de seu conteúdo. As chaves públicas podem ser compartilhadas por diversos meios diferentes, por exemplo, diretórios públicos disponíveis na Internet.

Qualquer mensagem cifrada com uma chave pública só poderá ser decryptografada por meio da chave privada correspondente.

Tome-se agora como referência um novo cenário: Alice é a autora de uma mensagem cifrada com sua chave privada e a seguir enviada a um ou mais destinatários.

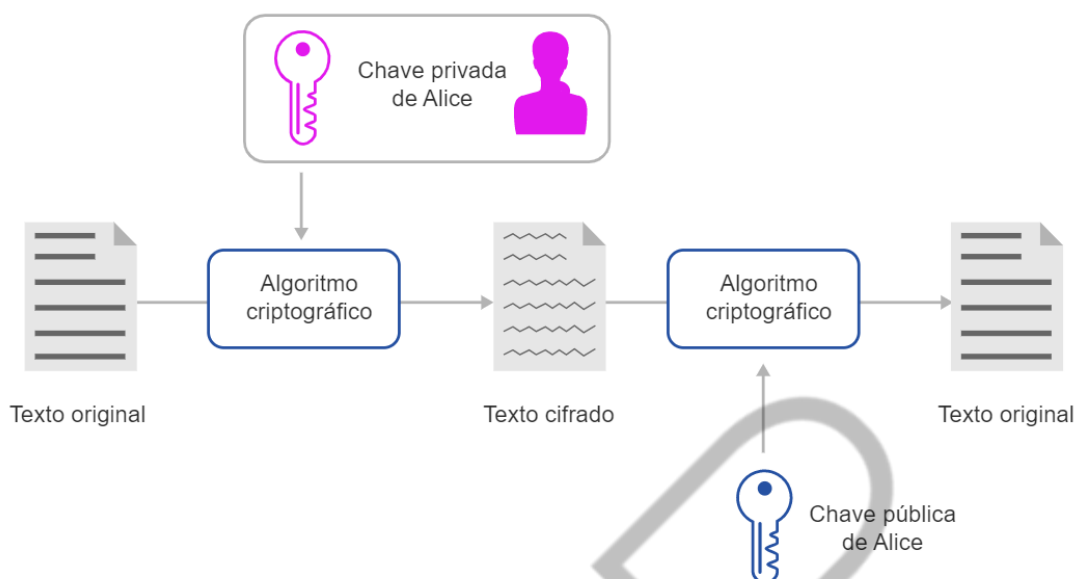


Figura 12 – Garantia da AUTENTICIDADE
Fonte: Cartilha de Certificação Digital (2018)

Tendo cifrado a mensagem com sua chave privada, Alice garante a AUTENTICIDADE dela, ou seja, a autoria de um documento ou a identificação em uma transação. Pode-se concluir, então, que a CONFIDENCIALIDADE e a AUTENTICIDADE utilizam os mesmos pares de chaves (pública e privada), porém, em sentidos inversos.

O destinatário de uma mensagem cifrada com uma chave privada só poderá decryptografá-la por meio da chave pública correspondente.

A criptografia assimétrica é baseada em funções matemáticas unidirecionais que impedem a recuperação da mensagem de entrada com base no resultado obtido na saída (ou ao menos, dificulta tanto tal procedimento que o torna computacionalmente inviável).

7 ALGORITMOS CRIPTOGRÁFICOS

Existem vários algoritmos criptográficos simétricos e assimétricos, desenvolvidos para aplicação aos mais variados contextos.

7.1 Algoritmos criptográficos simétricos

7.1.1 DES e 3DES

Algoritmo desenvolvido nos anos 1970 pelo NIST (National Institute of Standards and Technologies) em conjunto com a IBM. O DES (*Data Encryption Standard*, ou em português, padrão de encriptação de informação) é uma cifra de bloco, ou seja, um bloco de entrada em texto claro – no caso com tamanho de 64 bits, é tratado como um todo a fim de produzir na saída, um bloco cifrado de mesmo tamanho. Entretanto, como a chave criptográfica do DES é comprimida, ela passa a ter tamanho de 56 bits sendo, portanto, considerado inseguro pelos padrões atuais.

No 3DES (*Triple Data Encryption Standard*, cuja tradução é padrão de criptografia de dados triplo), o algoritmo DES é sequencialmente aplicado três vezes, com três chaves distintas de 56 bits, o que produz uma chave final com comprimento efetivo de 168 bits. No 3DES, a mensagem é cifrada pela primeira chave (K1), o resultado é decifrado pela segunda chave (K2), sendo então novamente cifrado pela terceira chave (K3), conforme ilustrado pela figura abaixo.

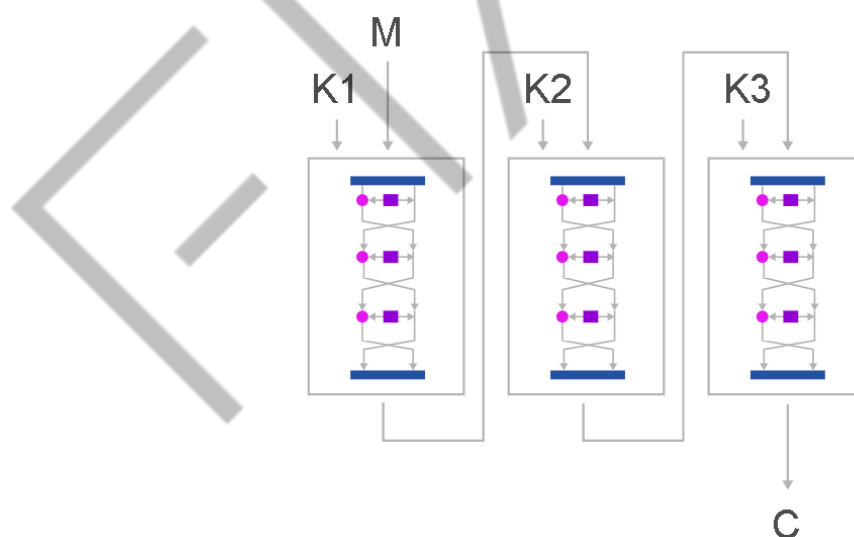


Figura 13 – Algoritmo 3DES
Fonte: Wikipedia.org (s.d.)

É possível ainda, em vez de utilizar-se de três chaves, a execução do processo utiliza apenas duas quando, então, a mensagem inicial é cifrada com a primeira chave, o resultado é decifrado pela segunda e novamente cifrado com a primeira, tendo-se assim, uma chave com tamanho efetivo de 112 bits.

7.1.2 RC2, RC4 e RC5

Os RCs (*Rivest Ciphers* ou, em português, cifras ricas) são uma sucessão de algoritmos que possuem maior flexibilidade e possibilitam maior segurança do que o DES simples. Todos são algoritmos simétricos (a mesma chave é usada para cifrar e para decifrar) (MOREIRA, 2002).

A exemplo do DES, o RC2 também é uma **cifra de blocos** que opera com entradas com 64 bits, podendo, entretanto, trabalhar com chaves criptográficas com até 2048 bits, o que pode torná-lo bem mais seguro que o DES.

O tamanho mais usual para a chave criptográfica é de 128 bits, o qual já ainda pode ser considerado como relativamente adequado. O RC2 pode ser um bom substituto para o DES, sendo em softwares, aproximadamente duas vezes mais rápido que ele.

Diferentemente dos dois anteriores, o RC4 é uma **cifra de fluxo**, o que significa que o fluxo de dados em texto claro é cifrado de forma contínua (*bit a bit, byte a byte* ou outras unidades de dados) com a ajuda de uma chave inserida por um gerador pseudoaleatório. Como o RC2, o RC4 também pode trabalhar com chaves de tamanho variável (máximo 2048 bits, tipicamente 128 bits) podendo, em softwares, vir a ser até dez vezes mais rápido que o DES.

O RC5 é também uma técnica de cifragem em bloco, conhecido por sua flexibilidade e possibilidade de parametrização. No RC5 os blocos de entrada podem ter qualquer tamanho predeterminado (32, 64 ou 128 bits), o usuário pode definir o tamanho da chave (0-255 Bytes), sendo possível ainda, determinar o número de iterações do algoritmo (0-255).

7.1.3 Idea (*International Data Encryption Algorithm*)

O IDEA também é uma técnica de cifragem simétrica em bloco, a qual utiliza-se de blocos fixos com 64 bits e chaves com tamanho **fixo** de 128 bits, fato que poderá se tornar desfavorável no futuro.

7.1.4 Blowfish

Algoritmo conhecido por sua velocidade, sendo mais rápido que o RC2 e o IDEA. Também trabalha com blocos fixos de 64 bits, mas as chaves podem ter qualquer comprimento, tipicamente, 128 *bits*.

7.1.5 AES

O AES (*Advanced Encryption Standard*, ou em português, padrão de criptografia avançada) nasceu de uma competição promovida pelo NIST para criação de um novo algoritmo que substituísse o DES, sendo o processo seletivo iniciado em 1997 e finalizado em 2000 com a vitória do algoritmo Rijndael, escrito por Vincent Rijmen e Joan Daemen.

O AES também é um algoritmo de bloco, cujo tamanho é fixo em 128 bits podendo, entretanto, o tamanho da chave criptográfica variar entre 128, 192 e 256 *bits*. Vale destacar que o algoritmo Rijndael e o AES não são exatamente iguais, uma vez que no primeiro, também os tamanhos de bloco podem ser especificados variando de 32 em 32 *bits* com tamanho mínimo de 128 *bits* e máximo de 256 *bits*. A tabela abaixo resume as principais características dos algoritmos referidos.

Algoritmo	Tipo	Tamanho da chave
DES	Por bloco	56
IDEA		128
RC2, RC5		1 a 2048
3DES		112 ou 168
AES		128, 192 ou 256
BLOWFISH		Tipicamente 128
RC5	Por fluxo	Definido pelo usuário

Tabela 2 – Principais características dos algoritmos criptográficos
Fonte: Elaborada pelo autor (2018)

7.2 Algoritmos criptográficos assimétricos

Diferentemente dos algoritmos simétricos, os assimétricos – também conhecidos como algoritmos de chave pública –, conseguem lidar de forma adequada com o gerenciamento das chaves, as quais passam agora, a ser duas: uma pública e outra privada.

7.2.1 Algoritmo de Diffie-Hellman

Este algoritmo tem como objetivo garantir que dois interlocutores possam trocar, de maneira segura, a chave (simétrica) a ser utilizada para cifragem das mensagens trocadas entre si.

A figura a seguir ilustra o processo de troca de chaves.

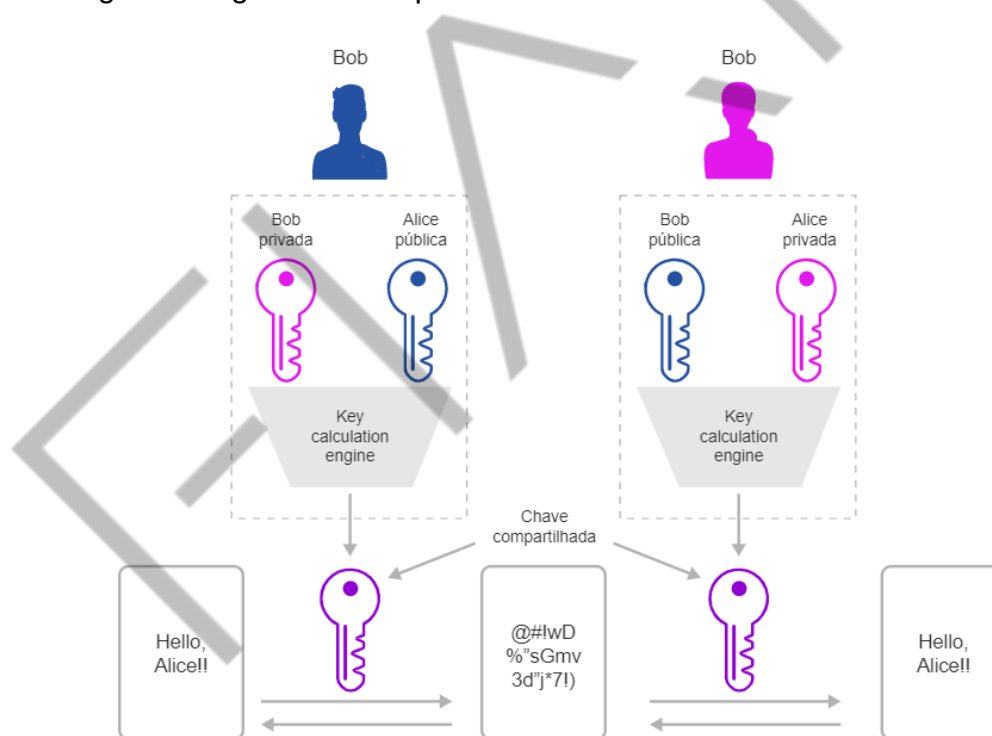


Figura 14 – Algoritmo de Diffie-Hellman

Fonte: Adaptada pelo autor (2018)

Na figura, Bob recebe cópia da chave pública de Alice, à qual é combinada sua chave privada, gerando, assim, uma chave compartilhada. Da mesma forma, Alice recebe uma cópia da chave pública de Bob e a combina à sua chave privada, obtendo assim, a mesma chave compartilhada. É importante salientar que no algoritmo de Diffie-Hellman, a combinação da chave privada A e da chave pública B **gera o mesmo**

resultado que a combinação da chave privada B e da chave pública A, e ainda, que ele pode ser usado para distribuição de chaves, mas não pode ser usado para criptografar/decriptografar as mensagens.

7.2.2 RSA

Algoritmo inventado por Ron Rivest, Adi Shamir e Leonard Adleman, o RSA tem sua segurança diretamente relacionada à dificuldade de realizar fatorações, utilizando-se de grandes números primos (acima de 100 dígitos). O RSA pode assegurar a confidencialidade e a autenticidade da mensagem, entretanto, é considerado um algoritmo lento, inadequado à cifragem de grandes blocos de dados.

7.2.3 SSL / TLS

O *Secure Socket Layer* (SSL), desenvolvido em 1994 pela Netscape e RSA, estabelece um canal criptografado entre o servidor web e o navegador do cliente, a fim de garantir a PRIVACIDADE, INTEGRIDADE e AUTENTICIDADE dos dados trafegados.

Na primeira fase da comunicação, cliente e servidor verificam quais os algoritmos suportados por ambos que serão utilizados na comunicação (RSA, DAS, ECDSA). O cliente requisita a um servidor que suporte o protocolo SSL uma conexão segura, prontamente enviando uma lista com os algoritmos disponíveis para a cifragem e autenticação dos dados.

Na segunda fase, o servidor recebe a requisição e escolhe o mais seguro dentre os algoritmos listados pelo cliente, que ele também possua, e o avisa dessa decisão. Ambos trocam chaves e realizam a autenticação – são utilizados algoritmos de chave pública (RSA e Diffie-Hellman, dentre outros). Para se autenticar, o servidor envia sua identificação na forma de um certificado digital, o qual geralmente contém o nome do servidor, a Autoridade Certificadora para verificação e sua chave pública.

Na terceira fase, as mensagens são autenticadas por códigos gerados por funções *hash* HMAC-MD5 ou HMAC-SHA, garantindo, a partir daí, que as mensagens sejam trocadas com segurança após passarem pelo SSL Record Protocol. Geralmente, a autenticação do servidor é feita por intermédio de uma Autoridade

Certificadora (AC). Nesse caso, o cliente se vale de uma chave pública da própria AC para validar sua assinatura no site do servidor. A AC deve estar na lista de ACs confiáveis para ter certeza de que o servidor é quem ele diz ser. O TLS é o nome adotado pela IETF desenvolvimento de um protocolo de segurança padronizado baseado no SSL 3.0.

Há algumas pequenas diferenças entre o SSL e o TLS, entretanto, o protocolo permanece substancialmente o mesmo – tanto que o protocolo TLS 1.0 é por vezes identificado como SSL 3.1. É comum encontrar aplicações que suportam ambos (SSL/TLS), não sendo, todavia, os dois protocolos interoperáveis, ou seja, somente um deles deve ser escolhido no momento da negociação. O TLS vem substituindo o SSL, especialmente em projetos envolvendo servidores *open source*, já entre os clientes, o SSL3 costuma ser mais difundido em detrimento do TLS.

7.2.4 DSA

O algoritmo de assinatura digital (DSA - *Digital Signature Algorithm*) foi desenvolvido pelo governo norte-americano para assinaturas digitais. O DSA pode ser usado exclusivamente para assinar dados e não para cifrá-los. O processo de assinatura do DSA recai sobre uma série de cálculos baseados em um número primo selecionado.

Embora projetado para usar chaves com tamanho máximo de 1024 bits, chaves mais longas são agora suportadas. Quando o DSA é utilizado, o processo de criação da assinatura digital é mais rápido do que sua validação. Quando o RSA é utilizado, o processo de validação da assinatura digital é mais rápido do que a criação.

8 ALGORITMOS DE *HASHING*

Conforme Oliveira (2012), a assinatura digital obtida por meio do uso da criptografia assimétrica ou de chave pública infelizmente não pode ser utilizada, na prática, de forma isolada, é necessário o emprego de um mecanismo fundamental para o adequado uso da assinatura digital. Esse mecanismo é a função *hashing*.

Assim, na prática, é inviável e contraproducente utilizar puramente algoritmos de chave pública para assinaturas digitais, principalmente quando se deseja assinar grandes mensagens, que podem levar preciosos minutos ou mesmo horas para serem integralmente cifradas com a chave privada de alguém, em vez disso é empregada uma função *hashing*, que gera um valor pequeno, de tamanho fixo, derivado da mensagem que se pretende assinar, de qualquer tamanho, para oferecer agilidade nas assinaturas digitais, além de integridade confiável.

Serve, portanto, para garantir a integridade do conteúdo da mensagem que representa, por isso, após o valor *hash* de uma mensagem ter sido calculado por meio do emprego de uma função *hashing*, qualquer modificação em seu conteúdo – mesmo em apenas um bit da mensagem – será detectada, pois um novo cálculo do valor *hash* sobre o conteúdo modificado resultará em um valor *hash* bastante distinto. É justamente por isso que a criptografia adotada para uma senha digitada (que nada mais é que um texto) é a função *hashing*, uma vez que esta garante a integridade da senha utilizada pelo usuário, além de assegurar o controle de acesso, uma vez que o texto em claro da senha não será compartilhado entre outros sistemas.

Seguem algumas características da função *hash*:

- **Unidirecionalidade:** Conhecida um resumo $h(M)$, deve ser computacionalmente impossível encontrar M a partir do resumo.
- **Compressão:** A partir de uma mensagem de qualquer longitude, o resumo $h(M)$ deve ter uma longitude fixa. O normal é que a longitude de $h(M)$ seja menor do que a da mensagem M .
- **Facilidade de cálculo:** Deve ser fácil calcular $h(M)$ a partir de uma mensagem M .
- **Difusão:** O resumo $h(M)$ deve ser uma função complexa de todos os bits da mensagem “ M ”, se modifica um só bit da mensagem M , o *hash* $h(M)$ deveria mudar a metade dos seus bits aproximadamente.
- **Colisão:** Será computacionalmente impossível, conhecido M , encontrar outro M' tal que $h(M) = h(M')$. Isso é conhecido como resistência débil às colisões.

Exemplos de algoritmos de cálculo de *hash*: md4, md5, sha1, sha256, sha512.

Conforme já anteriormente referido, estes algoritmos são utilizados para verificar a integridade da mensagem, certificando-se que mudanças imprevistas não tenham acontecido. A tabela a seguir mostra as principais características dos algoritmos de *hashing* considerados seguros pelo *Federal Information Processing Standards* (FIPS).

Algoritmo	Tamanho da mensagem (<i>bits</i>)	Tamanho do bloco (<i>bits</i>)	Tamanho da palavra (<i>bits</i>)	Tamanho do message-digest (<i>bits</i>)
SHA-1	$<2^{64}$	512	32	160
SHA-224	$<2^{64}$	512	32	224
SHA-256	$<2^{64}$	512	32	256
SHA-384	$<2^{128}$	1024	64	384
SHA-512	$<2^{128}$	1024	64	512
SHA-512/224	$<2^{128}$	1024	64	224
SHA-512/256	$<2^{128}$	1024	64	256

Tabela 3 – Propriedades dos algoritmos de *hash* seguros
Fonte: Adaptada do FIPS (2018)

Embora o md5 não figure na lista, ele ainda é um dos algoritmos de *hash* mais utilizados, com *message-digest* de 128 bits. A figura abaixo ilustra a extração dos *hashes* md5, sha1 e sha256 do arquivo `/var/log/messages` no Linux.

```

root@debian01:~# md5sum /var/log/messages
aec5a719313d9a874d52a4ba7f24806c /var/log/messages
root@debian01:~# shasum /var/log/messages
47c564c8f689d7d46346dee32bcd4f88e5962557 /var/log/messages
root@debian01:~# sha256sum /var/log/messages
ed521562eaa95316d2e159ff11ad20e26f19901b404aa6e800a1fa8fc81302276 /var/log/messages
root@debian01:~#

```

Figura 15 – *Hashes* do arquivo `/var/log/messages`
Fonte: Elaborada pelo autor (2018)

Da figura pode-se observar que o *hash* (*message-digest*) md5 do arquivo `/var/log/messages` é composto por 32 dígitos hexadecimais, totalizando assim 128 bits

(32x4=128). Da mesma forma, o *hash* sha1 do mesmo arquivo é composto por 40 dígitos hexadecimais, totalizando assim 160 bits (40x4=160) conforme apontado na tabela “Propriedades dos algoritmos de *hash* seguros”.

Digamos que alguém tenha feito uma cópia de um arquivo sigiloso, porém, com nome diferente do arquivo original a fim de mantê-la incógnita. Ao se extrair e comparar os *hashes* de ambos os arquivos, será possível atestar inequivocamente se tratar do mesmo arquivo sem, entretanto, abrir nenhum dos dois, já que o conteúdo é sigiloso. Ao se atribuir uma senha ao usuário, o *hash* é extraído e armazenado no sistema, em vez da própria senha. Isso evita que sua senha venha a ser facilmente revelada, caso o arquivo onde se encontra armazenado venha a vazar.

Há vários *sites* na Internet especializados em “*crackear*” *hashes* para recuperação das respectivas senhas, como o HashKiller (<https://hashkiller.io/listmanager>) dentre outros, bem como, softwares com a mesma finalidade, a exemplo do HashCat e John the Ripper.

O Hash_ID (<https://code.google.com/archive/p/hash-identifier/downloads>) é um pequeno *script* em *python*, escrito por Zion3R, licenciado sob o modelo GPL3, capaz de identificar os mais diversos tipos de *hashes*, podendo servir como auxiliar em ataques por *brute force*. Para fins de teste, foi aberto um terminal, como usuário desprivilegiado, na máquina virtual Debian01 já anteriormente utilizada, e digitado o comando:

```
user1@debian01:~$ wget https://storage.googleapis.com/google-code-archive-downloads/v2/code.google.com/hash-identifier/Hash_ID_v1.1.py
```

Comando de prompt 1 – Baixando o Hash ID usando o comando wget
Fonte: Elaborado pelo autor (2018)

Esse comando faz o *download* do Hash_ID, valendo ressaltar aqui que, sendo ele um *script* em *python*, poderá ser executado em qualquer plataforma que tenha essa linguagem instalada. A figura a seguir ilustra o *download* da ferramenta.

```

fiap@debian:~$
fiap@debian:~$ wget https://storage.googleapis.com/google-code-archive-downloads/v2/code.google.com/hash-identifier/Hash_ID_v1.1.py
--2020-03-23 14:07:26-- https://storage.googleapis.com/google-code-archive-downloads/v2/code.google.com/hash-identifier/Hash_ID_v1.1.py
Resolvendo storage.googleapis.com (storage.googleapis.com)... 216.58.202.176
Conectando-se a storage.googleapis.com (storage.googleapis.com)[216.58.202.176]:443... conectado
A requisição HTTP foi enviada, aguardando resposta... 200 OK
Tamanho: 34480 (34K) [application/octet-stream]
Salvando em: "Hash_ID_v1.1.py"

Hash_ID_v1.1.py      100%[=====] 33,67K  --.-KB/s   em 0,05s

2020-03-23 14:07:34 (652 KB/s) - "Hash_ID_v1.1.py" salvo [34480/34480]

fiap@debian:~$ █

```

Figura 16 – Hashes do Hash_ID.
Fonte: Elaborada pelo autor (2018)

Para se observar o funcionamento do Hash_ID, serão gerados os *hashes* MD5 e SHA1 da *string* de referência Cyb3r-S3cur1ty. No Linux a extração desses *hashes* é muito simples, e são utilizadas duas ferramentas nativas: md5sum e sha1sum. No terminal aberto na VM Debian01, basta proceder conforme ilustrado pela figura que segue.

```

fiap@debian:~$ echo Cyb3r-S3cur1ty | md5sum
64d5fa200080a48d7eb7a908bf909e77 -
fiap@debian:~$ echo Cyb3r-S3cur1ty | sha1sum
1c75b063e0887beb209a894f5d78462ceac41917 -
fiap@debian:~$ █

```

Figura 17 – Extração do *hash* da *string*
Fonte: Elaborada pelo autor (2018)

No primeiro *prompt* da figura acima, observa-se a linha:

```
user1@debian01:~$ echo Cyb3r-S3cur1ty | md5sum
```

Comando de prompt 2 – Comando *echo* com md5sum
Fonte: Elaborado pelo autor (2018)

O comando *echo* imprime na tela o argumento a ele passado, no caso, a cadeia de caracteres (ou *string*) Cyb3r-S3cur1ty. Entretanto, a intenção não é que esta *string* seja impressa na tela, mas sim, que seja passada ao comando md5sum, para que seja extraído seu *hash* MD5 (como se fosse uma senha digitada pelo usuário e armazenada pelo sistema), o que é feito por intermédio do *pipe* (|).

Analogamente, procede-se em relação à extração do *hash* SHA1, obtendo-se os resultados observados na figura. Para verificar se o Hash_ID consegue identificar o algoritmo utilizado, por meio do *hash* gerado, basta executá-lo, por meio da linha:

```
user1@debian01:~$ python Hash_ID_v1.1.py
```

Comando de prompt 3 – Hash_ID no *python*
Fonte: Elaborado pelo autor (2018)

Ao surgir a interface do Hash_ID, é fornecido o *hash* a ser verificado conforme ilustrado pela figura “*Extração do hash da string*”, e o *script* o analisa de imediato, trazendo uma lista dos algoritmos com maior probabilidade de haver gerado o *hash*.

[illegible]

Figura 18 – Verificação do *hash* MD5
Fonte: Elaborada pelo autor (2018)

Pela figura acima pode ser observado que fornecendo ao Hash_ID o *hash* MD5 extraído da *string* de referência, o *script* imediatamente o reconhece como tal. Entretanto, embora o resultado esteja correto, sempre há um certo grau de incerteza no resultado assim, o que faz com que o Hash_ID elenque outros algoritmos com boa probabilidade de haverem gerado o *hash* testado.

A figura abaixo ilustra o teste feito com o *hash* SHA1 da *string* de referência.

linhas) – sem espaços, enquanto o texto cifrado é lido verticalmente (em colunas), a partir da coluna cuja letra da chave seja a mais baixa.

M	I	L	K	W	A	Y
5	2	4	3	6	1	7
a	c	r	i	p	t	o
g	r	a	f	i	a	q
a	r	a	n	t	e	a
c	o	n	f	i	d	e
n	c	i	a	l	i	d
a	d	e	d	a	m	a
n	s	a	g	e	m	e
n	v	i	a	d	a	a
o	d	e	s	t	i	o
a	t	a	r	i	o	a

Figura 20 – Cifragem de transposição
Fonte: Elaborada pelo autor (2018)

Tome-se como exemplo, a mensagem: “a criptografia garante a confidencialidade e a autenticidade da mensagem enviada ao destinatário”, a ser cifrada com a chave MILKWAY – vide figura acima, com base na qual, obtém-se a mensagem cifrada:

Taedimmaicrocdsvdtifnfadgasrraani
eaieaagacnannoapitilaedtiogaedeeana

9.2 Base64

Base64 é um método para codificação de dados para transferência na Internet, tendo recebido esse nome em alusão ao conjunto de 64 caracteres ([A-Z],[a-z],[0-9], "/" e "+") que o compõe. O carácter "=" é utilizado como um sufixo especial, tendo a especificação original (RFC 989) definido que o símbolo "*" poderá ser utilizado para delimitar dados convertidos, mas não criptografados, dentro de um *stream*.

O *shell script* ilustrado pela figura abaixo demonstra de forma bastante simples a utilização da ferramenta **base64** do Linux, para codificação e decodificação de mensagens em base64.

```

1 #!/bin/bash
2 #
3 clear
4 echo -e "_____ BASE64 _____\n"
5 echo "Opcoes"
6 echo "1. Codificar input em base64"
7 echo "2. Decodificar input em base64"
8 echo -e "3. Sair\n"
9 read -p ">> " opt
10 #
11 case $opt in
12 1) read -p "Input a codificar: " codinp
13     echo $codinp | base64
14     read -p "Tecle algo para recomencar" i
15     ./codif64.sh ;;
16 #
17 2) read -p "Input a decodificar: " decodinp
18     echo $decodinp | base64 -d
19     read -p "Tecle algo para recomencar" i
20     ./codif64.sh ;;
21 #
22 3) echo "Script finalizado" ;;
23 esac

```

Figura 21 – *Shell script* para codificação/decodificação em base64
 Fonte: Elaborada pelo autor (2018)

Ao ser executado, o *shell script* ilustrado na figura acima solicita ao usuário que forneça a mensagem a ser codificada (linhas 12 a 15), na linha 13 observa-se que a saída do comando *echo* é redirecionada, por meio do *pipe* (“|”), para a entrada do comando *base64* fazendo com que o *input* fornecido pelo usuário do *script* seja codificado (veja figura “Codificação do *input* do usuário”).

```

_____ BASE64 _____

Opcoes
1. Codificar input em base64
2. Decodificar input em base64
3. Sair

>> 1
Input a codificar: codificacao64
Y29kaWZpY2FjYW82NAo=
Tecle algo para recomencar

```

Figura 22 – Codificação do *input* do usuário
 Fonte: Elaborada pelo autor (2018)

Da figura acima é possível observar que a *string* “codificacao64” foi codificada pelo *script*, apresentando na saída o resultado **Y29kaWZpY2FjYW82NAo=**. Na linha 18 do *script* é possível observar que o comando *base64* vem acompanhado pela opção *-d*, a qual indica que a entrada do usuário foi feita em *base64* e deve ser decodificada.

Mais informações a respeito do comando *base64* poderão ser obtidas diretamente no *prompt* do Linux, digitando-se: **man base64**. Nas linhas 15 e 20,

`./codif64.sh` é o nome atribuído ao *script*. Tendo, portanto, essas linhas como função, será reiniciado assim que o usuário digitar algo conforme solicitado após a codificação ou decodificação da mensagem.

EMEND

REFERÊNCIAS

CABRAL, C; CAPRINO, W. **Trilhas em segurança da informação** – Caminhos e ideias para proteção de dados. Rio de Janeiro: Brasport, 2016.

ETHICAL Hacking – Sniffing. Tutotials Point, [s.d.]. Disponível em: <https://www.tutorialspoint.com/ethical_hacking/ethical_hacking_sniffing.htm>. Acesso em: 4 mar. 2021.

FIPS. **Secure Hash Standard (SHS)**. [s.d.]. Disponível em <<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>>. Acesso em: 4 mar. 2021.

GTA UFRJ. **SSL & TLS**. Disponível em: <https://www.gta.ufrj.br/grad/11_1/tls/>. Acesso em: 4 mar. 2021.

IUS MENTIS. **Message digest / cryptographic hash functions**. [s.d.]. Disponível em: <<http://www.iusmentis.com/technology/hashfunctions/>>. Acesso em: 4 mar. 2021.

MORAES, A.F. **Segurança em Redes**. Fundamentos. São Paulo:Érica. 2013.

MOREIRA, A. Criptografia. **Instituto Superior de Engenharia do Porto**, 2002. Disponível em: <<https://www.dei.isep.ipp.pt/~asc/doc/criptografia.html>>. Acesso em: 03 mar. 2023.

OLIVEIRA, R. R. **Criptografia Simétrica e Assimétrica**: os principais algoritmos de cifragem. 2012. Disponível em: <<http://www.ronielton.eti.br/publicacoes/artigorevis-tasegurancadigital2012.pdf>>. Acesso em: 4 mar. 2021.

SVETONI Tranquilli Vita Divi Ivli. [s.d.]. Disponível em: <<http://thelatinlibrary.com/suetonius/suet.caesar.html#56>>. Acesso em: 4 mar. 2021.

TECHGNOR. **Man in the middle attack**. 2017. Disponível em: <<https://techgnor.blogspot.com/2017/10/man-in-middle-attack.html>>. Acesso em: 4 mar. 2021.

TENEMBAUM, A. S. **Redes de Computadores**. 5. ed. São Paulo: Pearson, 2016. E-book.

WALKER, M. **CEH Certified Ethical Hacker**. Exam Guide. USA: McGraw Hill, 2012.